

INTRODUCTION TO R & RSTUDIO

(Handbook on the use of R)

Introduction to R & RStudio

Basic syntax

R commands: basic syntax - expressions

R has a very simple syntax based on **expressions** or **assignments**, which are clearly stated within command lines.

Comments can be inputted anywhere, introduced by the '#' symbol: any content subsequent to the '#' symbol is disregarded and not executed (and is displayed **# in green**). An **expression** is **evaluated** by R, executed (if valid), and the outcome presented in the console, but is not saved. Examples of **expressions** (commands entered in a **script** or directly in the **console**):

```
3+5*3.5      # output: vector (with one element) numeric
log(3+5*3.5)
3>0          # output: vector (with one element) logical (TRUE/FALSE)
```

Output (**console**):

```
> 3+5*3.5      # output: vector (with one element) numeric
[1] 20.5
> log(3+5*3.5)
[1] 3.020425
> 3>0          # output: vector (with one element) logical (TRUE/FALSE)
[1] TRUE
```

Expressions usually include **operators**, such as '+' (arithmetic operator), '>' (comparison operator) as well as **functions** like `log()`.

R commands: basic syntax - assignments

An assignment also involves the evaluation of an expression, but in addition it saves the resulting value within an R object. The assignment operators are "<-" or "=" :

```
# I use the function c() (concatenate) to create a numeric VECTOR.
x <- c(1,5,8,-1,0,-2,10)
# I use the c() function to create a character VECTOR.
s = c("F", "M", "F", "M", "F", "M", "F", "M", "F")
# I use the > operator to evaluate which elements of x are positive.
l <- (x>0)
# I define a numeric vector from x
y <- x+2
```

When naming objects in R, it is important to note that:

- Object names may contain all alphanumeric symbols (letters, A-Z and a-z, or numbers) but **no spaces are allowed** [The characters "." and "_" are also allowed but not as initials].
- When an object is assigned a name of an already existing object, R **overwrites** the existing object **without any notification**
- R is **case-sensitive** (it differentiates between lowercase and uppercase letters, treating **x** and **X** as distinct entities)

R commands: basic syntax - data type

Unlike expressions, which are displayed in the output but not saved, **objects created through assignment are saved** but not automatically displayed. To print an object, you need to either submit a command that contains only the name of the object or use the `print()` function

```
# I use the function c() (concatenate) to create a numeric VECTOR.
x <- c(1,5,8,-1,0,-2,10)
# I use the c() function to create a character VECTOR.
s <- c("F", "M", "F", "M", "F", "M", "F", "M", "F")
# I use the > operator to evaluate which elements of x are positive.
l <- (x>0)

x
print(s)
print(l)
```

Output ([console](#))

```
> x
[1] 1 5 8 -1 0 -2 10
> print(s)
[1] "F" "M" "F" "M" "F" "M" "F" "M" "F"
> print(l)
[1] TRUE TRUE TRUE FALSE FALSE FALSE TRUE
```

numerical

character

logical

Types of values

R commands: basic syntax - "null"/special values

The R language encompasses several specific values, known as «nulls» that can be assigned, generating from specific operations or encountered in imported data.

```
> x <- c(1,5,8,-1,0,-2,10)
> log(x)
[1] 0.000000 1.609438 2.079442 NaN -Inf NaN 2.302585
Warning message:
In log(x) : NaNs produced
```

NaN (Not A Number): this value results from certain computations, e.g. `log(-2)` or `sqrt(-1)` (square root of a negative number), or `(0/0)`

Inf (or **-Inf**) (infinity): this value is the outcome of numerical calculations that returns an extremely large result, such as the result of division by 0, `(3/0)`, or `log(0)`. It is important to note that **Inf** is indeed a **number**, even though its magnitude exceeds representation limits.

NA (Not Available): this value indicates a missing value. **NA** are commonly encountered when data is absent for particular cases, either due to non-response or unavailability.

NULL: this value denotes an **empty** set, without any content (**not to be confused with an NA** or a **NaN**), often results from expressions or functions with undefined arguments.

Operators

In the previous examples some objects were created using some of the **operators** available in R. For instance , "<-" is an **assignment** operator, + and * are *arithmetic* (sum and multiplication) operators. R provides three fundamental classes of operators : arithmetic, relational (comparison) and logical. The following is a subset of these (basic) operators:

Operator	Description	Input	Usage	Result
+	Addition	x<-11 ; y<-3	x + y	14
-	Subtraction		x - y	8
*	Multiplication		x * y	33
/	Division		x / y	3.6666667
^	Raise to Power		x^y	1331

Operator	Description	Input	Usage	Result
<	Lesser than	x<-11 ; y<-3	x < y	FALSE
>	Greater than		x > y	TRUE
<=	Lesser than or equal		x <= y	FALSE
>=	Greater than or equal		x >= y	TRUE
=	Equal		x == y	FALSE
!=	Different		x != y	TRUE

Operator	Description	Input	Usage	Result
!	Logical NOT	x<-11 ; y<-3	!(x>2)	FALSE
			!(y>5)	TRUE
&	Logical AND		(x>2) & (y>5)	FALSE
	Logical OR		(x>2) (y>5)	TRUE
			(x>2) (y>1)	TRUE

Arithmetic operators

(can only be applied to numerical values)

Relational (comparison) operators

(some applicable only to numerical values)

Logical operators

Functions

Functions allow to compute specific values, manipulate objects or create new ones.

Simplifying the more advanced aspects, it is important to note that most functions in R rely on a sequence of **arguments**, denoted within parentheses after the function's name, and these arguments are separated by commas. Arguments represent the necessary **input data** that must be provided to the function for its proper execution, and they can be explored using the **args()** function.

Example: **log()** function

```
> args(log)
function (x, base = exp(1))
```

The function computes the logarithm of **x** within the provided **base**. Note that the **base** argument possesses a **default** value of **base=exp(1)**. Thus, when the function is called without indicating a specific base, it calculates the natural logarithm of **x**:

```
> # equivalent commands
> log(3, base=exp(1)) # equivalent to log(3,exp(1))
[1] 1.098612
> log(3)
[1] 1.098612
```


Functions

Function **arguments** can be specified in several ways:

- ▶ by explicitly mentioning each argument's respective name (**named form**, which is **recommended to prevent errors**). In this case, arguments can be indicated in **any** sequence, not necessarily in the order displayed by the **args()** function.
- ▶ by listing arguments without naming them: in this case R assumes that the order corresponds to the arrangement presented in **args()**
- ▶ through a blend of the two approaches, with certain arguments specified by their names and others placed entered in their correct sequence.

```
log(x=10,base=2) # named form
log(10,2) # no name, in the correct order
log(base=2,x=10) # named form with inverted order
log(10,base=2) # first argument at its position
log(base=2,10) # named form for 'base', then 'x' in the expected order
```

Note that:

```
> log(base=10)
Error: argument "x" is missing, with no default
```

When an argument lacking a default is not provided when calling the function, an error will be generated.

Functions

Another example: the `print()` function

```
> args(print)
function (x, ...)
```

The `print()` function is equipped with two arguments.

- ▶ `x` is the object you want to print, and **it must necessarily be specified**;
- ▶ the second argument (*dots argument*, `...`), also called *ellipsis*, denotes an undefined set of supplementary arguments that can either be specified or passed to other functions.

For comprehensive details about the function and its additional arguments, you can use the `?print` command that will open the **Help** tab in RStudio (located within the lower-right multifunction window). Alternatively you can directly search for `print` within the **Help** tab

Introduction to R & RStudio

Data structures in R

Data structures in R

R provides the flexibility to define various **data structures**, each possessing distinct **structures** and, potentially, distinct **functions** that can be applied to them (*object-oriented programming paradigm*: the outcomes of certain functions are influenced by the attributes of the input objects).

Focusing on the R's fundamental data structures and excluding non-essential elements for the course's objectives, we can categorize them as follows:

- ▶ **Vectors**: set of elements sharing a **common data type** (such as all numeric, or all character, or all logical values)
- ▶ **Matrices**: set of elements **all of the same data type** (numeric, character, logical) organized in rows and columns
- ▶ **Factors**: vectors that reassign the values of another vector, often of varying types, by associating each value with a specific **level** (or category).
- ▶ **Dataframes**: set of **columns** (potentially) **of different types** (**vectors of different types** - numeric, character or logical - and/or factors)
- ▶ **Lists**: set of structures of **any type**, typically used to organize the outcomes of **complex functions**

In the following slides, we will focus on **vectors**, **factors**, constituents of **dataframes**. *Matrices* (used marginally in the course) will be only briefly introduced, while we will discuss *lists* later, referring to the output of specific functions

Vectors: creation

Vectors are the most elementary data structure in R, and consist of a set of elements **all of the same type** (numeric, character, logical).

To create a vector you can use, for **example** (other functions are available):

- ▶ The function `c()` that concatenates an arbitrary number of elements (possibly vectors).
- ▶ The `rep()` function that generates a vector of repeated elements for a specified number of repetitions

```
x <- c(1,5,8,0,1,10,3)
s1 <- rep("North",4)
s2 <- rep("South",3)
s <- c(s1,s2) # equivalent to s <- c(rep("North",4), rep("South",3))
```

Warning: when creating vectors with differing data types, R will coerce the vector into a single, unified data type.

```
> y <- c(1,3,11, "F", "M") # combination of numbers and characters
1] "1" "3" "11" "F" "M"
```

Vectors: attributes

The `length()` function provides the size of a vector, representing the count of elements within the vector.

The `names()` function allows to assign names to vector elements, or retrieve names that have been assigned earlier.

```
x <- c(1,5,8,0,1,10,3)
names(x) <- c("Berlin", "Paris", "Amsterdam", "Stockholm", "Lisbon", "Madrid", "Rome")
length(x)
x
names(x)
```

```
> length(x)
[1] 7
> x
      Berlin      Paris Amsterdam Stockholm      Lisbon      Madrid      Rome
         1         5         8         0         1         10         3
> names(x)
[1] "Berlin"      "Paris"        "Amsterdam"    "Stockholm"    "Lisbon"       "Madrid"
[7] "Rome"
```

Vectors: indexing and selection

To access specific elements within a vector and/or transform selected elements, you can use the `[]` **operator** following the vector's name, specifying as arguments the positions of the elements to be selected, their names (if assigned), or the condition(s) that the elements should fulfill:

```
x
x.3<-x[3] # third element
x.3
x.sub1<-x[c(1,4,6)] # elements with position 1,4,6
x.sub2<-x[c("Berlin", "Stockholm", "Madrid")] # selection using element names
x.sub1 # identical to x.sub2
x.sub3<-x[-c(1,4,5,7)] # elimination of elements with positions 1,4,5,7
x.sub3
```

```
> x
  Berlin      Paris Amsterdam Stockholm      Lisbon      Madrid      Rome
      1          5          8          0          1          10          3

> x.3
Amsterdam
      8

> x.sub1 # identico a x.sub2
  Berlin Stockholm      Madrid
      1          0          10

> x.sub3
  Paris Amsterdam      Madrid
      5          8          10
```

Vectors: indexing and selection

Elements of a vector can also be selected based on conditions defined on the vector itself or on another vector

```
x.sub4<-x[x>=5] # element selection >= 5
x.sub4 # identical to x.sub3
s
s.n <- s=="North"
s.n
x.sub5 <- x[s=="North"] # condition based on another vector
x.sub6 <- x[s.n]
x.sub5 # identical to x.sub6
# selective editing
x.new<-x
x.new[x<5]<-0 # selective editing
x.new
```

```
> s
[1] "North" "North" "North" "North" "South" "South" "South"
> s.n
[1] TRUE TRUE TRUE TRUE FALSE FALSE FALSE
> x.sub5 # identical to x.sub6
  Berlin      Paris Amsterdam Stockholm
    1         5         8         0
> x.new
  Berlin      Paris Amsterdam Stockholm  Lisbon  Madrid  Rome
    0         5         8         0      0      10      0
```


Factors

Factors are numeric vectors that associate each value from another vector (of any type) with specific **levels** (or categories). These levels can be defined by the analyst and enable straightforward reclassification and recoding of variable types. To retrieve the levels associated with a factor, the `levels()` function is used.

```
> char.1<-c("F","M","F","F","M","F","M","F","M","F")
> factor.1<-factor(char.1)
> factor.1
[1] F M F F M F M F M F
Levels: F M

> levels(factor.1)<-c("Female", "Male") # recode
> factor.1
[1] Female Male Male Female Female Female Male Male Female Male Female.
Levels: Female Male

# definition of labels being created
> factor(char.1,levels=c("M", "F"),labels=c("Male", "Female"))
[1] Female Male Male Female Female Female Male Male Female Male Female.
Levels: Male Female

> char.1A<-c(1,2,1,1,2,1,2,1) # numeric vector to indicate gender
> factor(char.1A,levels=c(1,2),labels=c("Female", "Male"))
[1] Female Male Male Female Female Female Male Male Female Male Female.
Levels: Female Male
```

Factors

Factors can also be used to assign identical labels to distinct values or to arrange the values of vectors in a specific order.

```
> char.3<-c("Low", "Medium", "High", "Medium", "High", "VeryLow", "High", "Medium",  
"Low")  
> factor.3<-factor(char.3)  
> factor.3 # Levels are in alphabetical order!  
[1] Low Medium High VeryLow High  
[8] Medium Low  
Levels: High Low Medium VeryLow  
  
> # assign labels by sorting the modes  
> factor.3<-factor(char.3,levels=c("VeryLow", "Low", "Medium", "High", "VeryHigh"))  
> factor.3 # Levels are now sorted correctly  
[1] Low Medium High Verylow High Medium Low  
Levels: Verylow Low Medium High VeryHigh  
  
> # grouping modes  
> factor.4<-factor3  
> levels(factor.4)  
[1] "VeryLow" "Low" "Medium" "High" "Very High"  
> levels(factor.4)<-c("Low", "Low", "Medium", "High", "High")  
> factor.4  
[1] Low Medium High Medium High Low Medium Low.  
Levels: Low Medium High
```

Matrices: creation

Matrices, much like vectors, consist of **elements sharing a common data type** (numeric, character, logical). However, unlike vectors which have a single dimension, matrices are structured in rows and columns, creating a two-dimensional arrangement.

To create a matrix, you can use the `matrix()` function, which requires arguments such as:

x (vector containing the elements of the matrix)

nrow and **ncol** (number of rows and number columns)

byrow (a logical value specifying whether the matrix should be filled by rows (**TRUE**) or by columns (**FALSE**; default)).

```
> num.v<-c(1,3,4.1,5,6.1,8,9,10.11,30,5.3)
> mat.num.1<-matrix(num.v,nrow=2,ncol=5) # matrix filled by columns
> mat.num.1
      [,1] [,2] [,3] [,4] [,5]
[1,]    1  4.1  6.1  9.00 30.0
[2,]    3  5.0  8.0 10.11  5.3
> mat.num.2<-matrix(num.v,nrow=2,ncol=5,byrow=T) # matrix filled by rows
> mat.num.2
      [,1] [,2] [,3] [,4] [,5]
[1,]    1    3  4.10    5  6.1
[2,]    8    9 10.11   30  5.3
```

Matrices: creation

Arrays can also be created using functions like `cbind()`, which aligns vectors (of the same length) side by side to form columns, or `rbind()` that stacks vectors (of the same length) atop one another to create rows.

```
> mat.num.2<-matrix(num.v,nrow=2,ncol=5,byrow=T) # matrix filled by rows
> mat.num.2
      [,1] [,2] [,3] [,4] [,5]
[1,]     1     3 4.10     5 6.1
[2,]     8     9 10.11    30 5.3

> num.2A<-c(1,3,4.1,5,6.1)
> num.2B<-c(8,9,10.11,30,5.3)
> mat.num.2<-rbind(num.2A,num.2B) # obtained by overlapping The two vectors by row

> mat.num.3<-matrix(c(1:3,21:23,1,120,240,5,9,12),nrow=3,ncol=4)
      [,1] [,2] [,3] [,4]
[1,]     1    21     1     5
[2,]     2    22    120     9
[3,]     3    23    240    12

> mat.num.3<-cbind(1:3,21:23,c(1,120,240),c(5,9,12)) # bind vectors
```

Similar to other data structures, arrays allow for actions such as determining their dimensions, assigning names to rows and columns, and selecting specific elements. Further details can be found in the R manual (they are similar to those used for dataframes).

Matrices: attributes [OPTIONAL]

The `dim()` function returns a vector with the dimensions of the array; the two functions `nrow()` and `ncol()`, on the other hand, provide the number of rows and columns in the array.

Similar to vectors, you can assign names to the rows and columns of the array using the `rownames()` and `colnames()` functions.

```
> mat.num.2
      [,1] [,2] [,3] [,4] [,5]
[1,]     1     3 4.10     5 6.1
[2,]     8     9 10.11    30 5.3
> dim(mat.num.2) # size of the matrix
[1] 2 5
> nrow(mat.num.2)
[1] 2
> ncol(mat.num.2)
[1] 5
> colnames(mat.num.2)<-c("X1", "X2", "Y1", "Y2", "Y3")
> rownames(mat.num.2)<-c("Ireland", "Sweden")
> mat.num.2
      X1 X2     Y1 Y2  Y3
Ireland 1  3  4.10  5 6.1
Sweden  8  9 10.11 30 5.3
```

Matrices: indexing and selection [OPTIONAL]

To extract a subset of elements from an array you can use the same selection operator introduced for vectors, `[]`, but inside the square brackets you will need to indicate the row and column indices of the desired elements, separated by a comma. If your intention is to extract all rows or columns from an array, you can omit specifying any index.

```
> mat.num.3<-matrix(c(1:3,21:23,1,120,240,5,9,12),nrow=3,ncol=4)
> rownames(mat.num.3)<-c("Subj1", "Subj2", "Subj3")
> colnames(mat.num.3)<-c("X1","X2","X3","X4")
> mat.num.3
      X1 X2  X3 X4
Subj1  1 21   1  5
Subj2  2 22 120  9
Subj3  3 23 240 12
> mat.num.3[,1] # selection first column
Subj1 Subj2 Subj3
     1      2      3
> mat.num.3[c(1,3),] # selection of row 1 and 3
      X1 X2  X3 X4
Subj1  1 21   1  5
Subj3  3 23 240 12
> mat.num.3[c(1,3),c("X1", "X2", "X4")] # selection of rows 1,3, columns 1,2,4
      X1 X2 X4
Subj1  1 21  5
Subj3  3 23 12
> # mat.num.3[1:2,-3] # selection of rows 1 and 2 and removing column 3
```

Dataframe

Dataframes are data structures typically used in R to manage datasets.

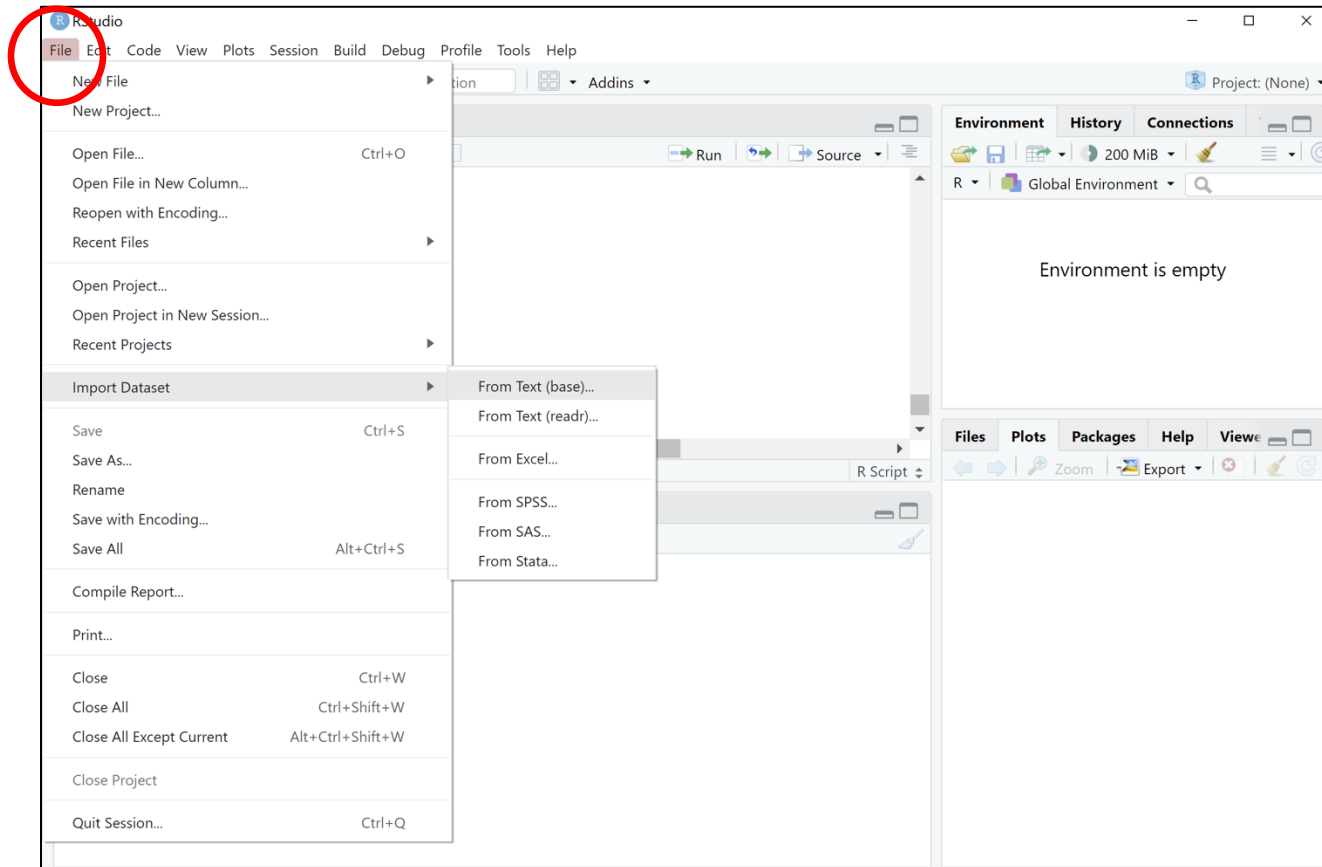
They take the form of tables with **observations presented in rows and variables presented in columns**. Dataframes can consist of **a blend of numeric, categorical, logical columns, and even factors**.

Although it is possible to create a dataframe from scratch (details in the R manual), usually (and also in the course) a dataframe is created:

- ▶ Importing data from alternate formats such as .csv, .xlsx,...
- ▶ Loading data into RStudio that was previously stored in a file generated by R, often with a .Rdata extension.

Dataframe: Importing data in text format (.csv)

To import a file from an **external source** (e.g., .csv or .xlsx format) you can select **Import dataset** in the **File** menu by selecting the appropriate file type to be imported. For example, to import a .csv file you would select **File->Import Dataset->From Text (base)**.



In Rstudio, a window will appear to guide you through the process of selecting import settings before the file is loaded into your R environment.

Dataframe: Importing data in text format (.csv)

Following the provided instructions you can import the Loyalty.csv file into RStudio. RStudio opens a window to allow you to choose how to import the file, before loading it.

The screenshot shows the 'Import Dataset' dialog box in RStudio. The 'Name' field is set to 'Loyalty'. The 'Input File' field shows the file path. The 'Encoding' is set to 'Automatic'. The 'Heading' is set to 'Yes'. The 'Row names' are set to 'Automatic'. The 'Separator' is set to 'Comma'. The 'Decimal' is set to 'Period'. The 'Quote' is set to 'Double (")'. The 'Comment' is set to 'None'. The 'na.strings' field is set to 'NA'. The 'Strings as factors' checkbox is unchecked. The 'Data Frame' preview shows the following columns: Customer, Age, Income, Payment, Dept, and Nr_v. The preview displays the first 17 rows of data.

Customer	Age	Income	Payment	Dept	Nr_v
1	35	[5-15]	Credit Card	Clothing	4
2	25	[15,20]	Cash	Clothing	4
3	54	[20,25]	ATM	Clothing	3
4	60	[25,35]	Cash	Clothing	7
5	44	[35,60]	ATM	Clothing	2
6	62	[15,20]	Credit Card	Clothing	2
7	27	[35,60]	Cash	Accessories	1
8	55	[20,25]	Cash	Accessories	3
9	62	[20,25]	Credit Card	Accessories	2
10	64	[20,25]	Cash	Accessories	2
11	55	[25,35]	Cash	Perfumery	3
12	58	[25,35]	Cash	Kids	4
13	49	[15,20]	Cash	Home/Textiles	3
14	59	[15,20]	Cash	Home/Textiles	2
15	70	[5-15]	ATM	Home/Textiles	2
16	46	[15,20]	ATM	Kids	3
17	42	[5-15]	ATM	Perfumery	4

Clicking the Import button finalizes the process and generates the **Loyalty** object within your R environment.

Dataframe: Importing data in text format (.csv)

The structure of this object (or any other object in R) can be displayed with `str()` function.

```
> str(Loyalty)
'data.frame':   125 obs. of  9 variables:
 $ Customer    : int  1 2 3 4 5 6 7 8 9 10 ...
 $ Age         : int  35 25 54 60 44 62 NA 55 ...
 $ Income      : chr   "[5-15)" "[15,20)" ...
 $ Payment     : chr   "Credit Card" "Cash" ...
 $ Dept        : chr   "Clothing" "Clothing" ...
 $ Nr_visits   : int   4 4 3 7 2 2 1 3 2 2 ...
 $ Amount      : num   263 243 274 585 152 ...
 $ Profitability: num   81.8 82.6 85.7 86.8 ...
 $ Recommend   : chr   "Neutral" "Neutral" ...
```

To view the first rows of a dataframe, you can use the `head()` function.

```
> head(Loyalty)
  Customer Age Income      Payment      Dept Nr_visits Amount Profitability      Recommend
1         1  35  [5-15) Credit Card Clothing         4   263.2         81.8         Neutral
2         2  25 [15,20)         Cash Clothing         4   243.2         82.6         Neutral
3         3  54 [20,25)         ATM  Clothing         3   273.6         85.7         Neutral
4         4  60 [25,35)         Cash Clothing         7   585.2         86.8         Neutral
5         5  44 [35,60)         ATM  Clothing         2   151.6         88.9 Very Unlikely
6         6  62 [15,20) Credit Card Clothing         2   102.0         91.5         Neutral
```

Dataframe: structure and attributes

The functions `dim()`, `nrow()`, and `ncol()` return a vector representing the size of the dataframe, the count of rows and the count of columns in the dataframe, respectively.

The two functions `rownames()` and `colnames()` allow to access or modify row and column names within the dataframe.

```
> dim(Loyalty) # dimension of the dataframe
[1] 125    9
> nrow(Loyalty)
[1] 125
> ncol(Loyalty)
[1] 9
> colnames(Loyalty)
[1] "Customer"      "Age"           "Income"        "Payment"
[5] "Dept"          "Nr_visits"     "Amount"        "Profitability"
[9] "Recommend"
> rownames(Loyalty)
[1] "1"  "2"  "3"  "4"  "5"  "6"  "7"  "8"  "9"  "10" "11"
[12] "12" "13" "14" "15" "16" "17" "18" "19" "20" "21" "22"
[23] "23" "24" "25" "26" "27" "28" "29" "30" "31" "32" "33"
...
```

Dataframe: indexing and selection

To extract a subset of elements from a dataframe, you can use the same selection operator `[]` for dataframes as you would for vectors. Within the square brackets you can specify the row and column indices of the elements to be extracted (or even the names of the rows and columns), separated by a comma. If you aim to extract all rows or all columns, you can omit specifying any index.

```
> Loyalty[3,4] # extraction of an element
[1] "ATM"
> Loyalty[3,] # select third line
  Customer Age  Income Payment      Dept Nr_visits Amount Profitability Recommend
3         3  54 [20,25)    ATM Clothing         3  273.6          85.7      Neutral
> head(Loyalty[,2]) # select second column/first elements
[1] 35 25 54 60 44 62
> # extraction first two rows and Payment column
> Loyalty[1:2, "Payment"]
[1] "Credit Card" "Cash"
> # extraction first 5 rows and some columns
> Loyalty[1:5,c("Customer", "Nr_visits", "Amount")]
  Customer Nr_visits Amount
1         1         4  263.2
2         2         4  243.2
3         3         3  273.6
4         4         7  585.2
5         5         2  151.6
```

Dataframe: selection and indexing

A convenient and concise way to select a single variable from a dataframe is by using the `$` symbol immediately after the dataframe's name, followed by the variable's name. This syntax is particularly useful for operations and for seamlessly adding new columns to the dataframe.

```
> Loyalty$Nr_visits # column Nr_visits
[1] 4 4 3 7 2 2 1 3 2 2 3 4 3 2 2 3 7 3 2 2 2 2 4 3 3 3 3 2 2 1 4 2 2 6 1 2 2 3
[40] 3 2 4 4 4 2 2 4 2 3 6 1 2 3 1 2 2 3 4 3 1 2 2 2 4 2 2 4 1 2 3 6 2 2 2 1 4 3 4
[79] 2 3 2 1 2 2 3 3 3 3 2 3 3 3 6 1 2 1 3 1 3 3 2 1 1 3 2 2 1 3 2 6 3 2 4 1 4 2 3
[118] 6 3 1 2 1 2 2 2
> # creation of new vars
> Loyalty$Amount_V<-Loyalty$Amount/Loyalty$Nr_visits
> Loyalty$Recommend.F<-factor(Loyalty$Recommend,
+ levels=c("Very Unlikely", "Unlikely", "Neutral",
+ "Likely", "Very Likely"))
> # operations on columns
> sum(Loyalty$Amount_V>100)
[1] 33
> mean(Loyalty$Amount_V>100) # % of customers with Amount_V>100
[1] 0.264
> mean(Loyalty$Age<=30) # missing value!!!
[1] NA
> mean(Loyalty$Age<=30,na.rm=T)
[1] 0.08064516
>
> Loyalty$Age gt 30<-Loyalty$Age>30
```

Dataframe: creation from scratch [OPTIONAL]

Although we will generally consider dataframes imported from external sources, it is possible to create a dataframe from scratch through the `data.frame()` function.

```
mat<-matrix(1:16,nrow=4)
vecnum <- c(1,10,12,20)
vecstring<-c("Male","Female","Female","Male")
fact<-factor(c(1,1,1,4))
levels(fact)<-c("North","South")
veclogic <- (vecstring=="Male" & fact=="North")
df1 <- data.frame(vecstring,fact,veclogic,vecnum,mat)
df1
```

```
> df1
  vecstring fact veclogic vecnum X1 X2 X3 X4
1     Male North    TRUE      1  1  5  9 13
2   Female North   FALSE     10  2  6 10 14
3   Female North   FALSE     12  3  7 11 15
4     Male  South   FALSE     20  4  8 12 16
```

Dataframe: structure and attributes [OPTIONAL]

As mentioned, the `dim()` function returns a vector with the size of the dataframe, and the two functions `nrow()` and `ncol()` provide the number of rows and columns.

Using `rownames()` and `colnames()` it is possible to assign or modify row and column names. However, it's important to note that these names can also be directly defined during the creation of the dataframe..

```
> dim(df1) # size of dataframe
[1] 4 8
> colnames(df1)
[1] "vecstring" "fact" "veclogic" "vecnum" "X1" "X2" "X3" "X4"
> colnames(df1)<-c("Gender","Region","Male.North","Y1","X1","X2","X3","X4")

> # dataframe creation by making column names explicit when creating
> df1 <- data.frame(Gender=vecstring, Region=fact, Male.North=veclogic,
+ Y1=vecnum, mat)
> df1
  Gender Region Male.North Y1 X1 X2 X3 X4
1   Male  North        TRUE  1  1  5  9 13
2 Female  North       FALSE 10  2  6 10 14
3 Female  North       FALSE 12  3  7 11 15
4   Male  South       FALSE 20  4  8 12 16
```

Lists: attributes [OPTIONAL]

The last type of structure we consider are **lists**. The final data structure we'll explore is the list. Lists are highly versatile objects that can contain elements of any type, including vectors, matrices, dataframes, functions, plots, and even other lists. They play a crucial role in organizing the outcomes of complex functions. Although we will not define lists in this course, you can construct a list using the `list()` function whose arguments are the desired list elements separated by commas.

```
# a list containing a dataframe, a factor, a numeric value, and a vector
list.1<-list(Loyalty, factor.1, log(3), c(1,2,5,6,7,8,11))
```

The number of elements within the list is returned by the `length()` function.

It is possible to assign names to the list's elements using the `names()` function and you can even create a list by directly naming its elements during creation.

```
> length(list.1)
[1] 4
> names(list.1)<-c("Dframe", "factor", "mynumber", "myvector")

> # list created by naming its elements
> list.2<-list(Dframe=Loyalty, factor=factor.1, mynr=log(3), myvec=c(1,2,5,6,7,8,11))
```


Lists: indexing and selection [OPTIONAL]

To access a specific element within a list, you can use the name of the list followed by the `[[...]]` operator providing the argument as either the position of the element in the list or its assigned name. It is also possible, just like with dataframes, to use the `$` symbol after the list's name, followed by the name of the element you wish to access.

```
> list.1[[2]] # extraction of the second element
$factor
[1] F M F F M F M F M F
Levels: F M
> list.1$myvector # extraction of element named myvector
[1] 1 2 5 6 7 8 11
```